Technical Documentation: Architecture & Implementation

Project Name: Inktellect: An AI Learning Buddy
Version: 1.0.0
Backend: Python 3.10+, Flask
Frontend: HTML5, Tailwind CSS, JavaScript (Lucide Icons)

1. System Architecture

Inktellect utilizes a **Client-Server Architecture** with **Stateful Session Management**. This ensures that user progress, preferences, and document contexts are maintained throughout the learning journey.

The Request-Response Cycle

Layer	Component	Function
Client	Tailwind / JS	Renders the UI and handles user interaction (AJAX requests).
Logic	Flask (app.py)	Manages routing, session validation, and text processing.
Intelligence	Groq & RAG	**Vector Search:** FAISS scans local embeddings. **Inference:** Llama 3.3 70B processes the augmented prompt.
Storage	Filesystem & Session	/uploads for docs; Signed Cookies for badges and progress.

2. The RAG Pipeline

The **RAGManager** class is the core engine, allowing the AI to access specific student documents without requiring model fine-tuning.

Pipeline Workflow

1. **Ingestion & Cleaning:** Parsed text from .pdf, .docx, or .txt is processed via Regex (_clean_extracted_text) to repair fragmented OCR data.

2. **Vectorization:**
    - **Chunking:** Split into 500-word blocks with a 50-word overlap.
    - **Embedding:** Converted into 384-dimensional vectors using `all-MiniLM-L6-v2`.
3. **Indexing:** Vectors are stored in a `FAISS IndexFlatL2`. Each `session_id` has an isolated index to ensure data privacy between users.

3. Module Logic & Key Functions

A. Knowledge Booster (Quiz Gen)

- **Method: Few-Shot Prompting** to enforce JSON outputs.
- **Validation:** A backend utility strips Markdown tags (e.g., ```` ```json ````) before passing data to `json.loads()` to prevent frontend crashes.

B. Gamification Engine

- **Function:** `check_badge_conditions(stats)`
- **Trigger:** Runs after file uploads, quiz completions, or chat interactions.
- **Persistence:** Earned badges are stored in the session and triggered on the frontend via **Toast notifications**.

C. Study Path Generator

- **Logic:** Performs temporal calculations by feeding the current date and user "Available Time" into the LLM to generate a chronological sequence of milestones.

4. Setup & Installation Guide

Prerequisites

- **Python:** 3.8+
- **Hardware:** 1GB+ RAM (required for the Sentence-Transformer model).
- **Credentials:** Groq API Key, Flask Secret Key.

Installation Steps

```
Bash
# 1. Clone & Navigate
git clone [repository-url]
cd inktellect

# 2. Environment Setup
python -m venv venv
source venv/bin/activate  # Windows: venv\Scripts\activate
```

```
pip install -r requirements.txt

# 3. Launch
python app.py
```

5. Recommended Documentation Screenshots

Screenshot Target	Purpose
RAGManager Class	Visualize the `search` and `add_document` methods in `app.py`.
Badge Logic	Show the `BADGES` dictionary and the condition-checking function.
File Management	Show the `/uploads` folder demonstrating `secure_filename` logic.
Network DevTools	Capture an AJAX request to `/api/chat` showing the JSON response.

6. Future Architectural Roadmap

- **PostgreSQL Integration:** Move from temporary sessions to persistent database storage.
- **Asynchronous Tasks:** Implement **Celery/Redis** for background document embedding.
- **Multi-Modal Support:** Integrate **Vision-LLMs** to interpret diagrams and charts within student PDFs.